

ML Engineer Take-Home: Hotel Attribute Extraction & Tiering

Context

We're building a **Hotel Curator** that extracts structured quality signals about hotels from unstructured review text and maps those signals to tier ratings (Elite / Superior / Premium / Fail) for use in a corporate travel booking product.

Hotel quality information is fragmented and inconsistent across sources. Review text, however, contains rich signals about specific attributes — room size, Wi-Fi quality, cleanliness, location accessibility, safety, and more — but extracting them reliably at scale requires a real ML pipeline.

Your task is to design and prototype that pipeline.

The problem

Given a corpus of hotel reviews, build a system that:

1. Extracts structured signals about a defined set of hotel attributes from review text.
2. Aggregates per-hotel signals into tier ratings (Elite / Superior / Premium / Fail, or your own scheme).
3. Surfaces evidence — which sentences drove each tier assignment.

You'll be evaluated on the **full pipeline** — data exploration, labeling strategy, modeling, evaluation, aggregation, and engineering craft — not just final accuracy numbers.

Dataset

Use the TripAdvisor Hotel Reviews dataset on Kaggle:

<https://www.kaggle.com/datasets/andrewmvd/trip-advisor-hotel-reviews>

You may augment with additional public data if useful — document any external data clearly.

Attributes to extract

Pick **5 attributes** from the list below — or propose your own with justification. Aim for a mix: include at least one numeric/factual attribute and at least two sentiment-based attributes.

- Room size (numeric — sq ft / m² when stated)
 - Wi-Fi quality (sentiment)
 - Cleanliness (sentiment)
 - Location accessibility (factual — proximity to landmarks / transit)
 - Safety & security (sentiment + factual)
 - Bed / sleep quality (sentiment)
 - Noise level (sentiment)
 - Staff service (sentiment)
 - Bathroom quality (sentiment)
 - Workspace suitability (factual + sentiment)
-

What we expect you to do

1. Data exploration & labeling strategy

- Explore the corpus — distribution, language quality, hotel coverage, review length, what's hard about this data.
- Define a label schema for your chosen attributes (sentence-level vs. doc-level, polarity granularity, multi-label vs. multi-class). Justify the trade-offs.
- Build a labeled dataset. You may use weak supervision, zero-shot models, LLM labeling, manual annotation, or a hybrid — your call. Justify the choice and report agreement / quality.

2. Modeling

- Train a model (or models) to predict your labels.
- Justify your modeling choices — architecture, loss, class imbalance handling, splits.
- We don't care about SOTA numbers. We care about correctness, calibration, and your

ability to reason about errors.

3. Aggregation to tiers

- Define a tier mapping for each chosen attribute (Elite / Superior / Premium / Fail or your own).
- Justify thresholds — they should be defensible, not arbitrary.
- Handle low-evidence hotels (few reviews, contradictory signals) explicitly.

4. Evidence / explainability

- For any hotel × attribute, your system should surface the sentences that drove the assignment.

5. Writeup (2–4 pages)

- What you built and why.
- Key design decisions and trade-offs.
- Evaluation methodology and results, with per-attribute breakdown.
- What you'd do with another week.
- Limitations, risks, and known failure modes.

Stretch (optional, only if time permits)

- A simple inference API returning JSON for `GET /hotels/{id}/attributes`.
- A minimal UI showing tiers + clickable evidence.
- An active learning loop — which examples should be labeled next, and why.
- A bias / fairness pass across hotel categories or geographies.

Time expectation

Plan for **6–8 hours**. We don't expect a production system — we expect clear thinking, defensible choices, and honest engineering. If you spend significantly more, that's fine but please note it in the writeup so we can calibrate.

Deliverables

1. **Git repository** (public, or private with access shared) containing:
 - All code, with a README explaining how to run it end-to-end.
 - Notebook(s) or scripts demonstrating the pipeline.
 - The labeled dataset you created (or a representative sample if size is an issue).
 - Trained model artifacts, or instructions to reproduce them.
 2. **Writeup** as `WRITEUP.md` or PDF in the repo.
 3. **Optional**: a 5–10 min Loom walkthrough of the repo.
-

Evaluation rubric

Area	What we look for
Problem framing	Clear understanding of the task; sensible scoping; pushing back where our framing is weak
Data work	Depth of exploration, sound labeling strategy, awareness of biases and noise
Modeling	Justified choices; not over-engineered; right tool for each attribute type
Evaluation	Right metrics; per-class analysis; honest reporting of weaknesses
Aggregation & tiering	Defensible thresholds; sane handling of low-evidence hotels
Code quality	Readable, structured, reproducible; tests where they matter
Communication	The writeup tells a story we can follow without diving into the code
Engineering judgment	Spotting trade-offs, pruning unnecessary work, naming limitations

What we will *not* penalize

- Modest accuracy numbers, if your methodology is sound.

- Using LLMs or pre-trained models — we want to see *smart* use, not lazy use.
 - Skipping stretch goals.
 - Disagreeing with our framing. If you think a different approach is better, say so in the writeup and do that instead.
-

Submission

Reply to this email with the repo link and writeup. Questions during the assignment are welcome — reach out any time.